# Assignment 3 Report: MNIST and Fashion-MNIST Classification/Regression using PCA, Linear Models, Trees, Bagging, and Random Forests

Roll Number: 2023090

## 1 Problem Statement

The objective of this assignment is to study the behavior of linear models and tree-based models on image classification and regression tasks. The work is divided into three parts: (i) ridge and lasso regression classification on MNIST digits $\{0, 1, 2\}$ after PCA reduction, (ii) decision trees, bagging, and random forests on the same MNIST subset, and (iii) a regression stump and bagging on Fashion-MNIST classes $\{0, 1, 2\}$ after PCA reduction.

## 2 Data Preprocessing

### 2.1 Filtering and Sampling

For all three questions, only the required classes were retained. For MNIST, classes 0, 1, and 2 were selected from the official train and test splits in `mnist.npz`. For Fashion-MNIST, classes 0, 1, and 2 were selected from the train and test splits.

### 2.2 Flattening and Normalization

Each image was flattened into a vector and normalized by dividing every pixel value by 255. Thus, the input matrix for each dataset was formed from normalized feature vectors.

### 2.3 Random Seed

All experiments were run with the fixed seed `2023090` for reproducibility.

## 3 Question 1: Ridge and Lasso Classification after PCA

### 3.1 Dimensionality Reduction

PCA was fitted on the complete training subset of MNIST digits $\{0, 1, 2\}$ and then applied to both the training and test sets. The main experiment used $p = 10$ PCA dimensions. For the complexity study, the ridge model was also evaluated for $p \in \{2, 5, 10, 20, 30\}$.

### 3.2 Target Encoding

For each class $k \in \{0, 1, 2\}$, a one-vs-rest target vector was defined as

$$y_k = \begin{cases} 1 & \text{if the sample belongs to class } k, \\ 0 & \text{otherwise.} \end{cases}$$

This produces three binary regression targets, one for each class.

### 3.3 Ridge Regression

For each class, ridge regression was fit using the PCA features. The ridge objective is

$$\frac{1}{n}\|y - Xw - b\|_2^2 + \lambda\|w\|_2^2,$$

where the intercept is left unregularized. The final class prediction was obtained using the highest regression score among the three class models.

### 3.4 Lasso Regression

Lasso regression was fit in the same one-vs-rest fashion. The implementation used `sklearn.linear_model.Lasso`, which was the only scikit-learn routine used in the assignment. The number of non-zero coefficients was recorded to measure sparsity.

### 3.5 Plots and What They Show

- Ridge and lasso training/test MSE versus $\lambda$.
- Number of non-zero lasso coefficients versus $\lambda$.
- Regularization paths for ridge and lasso for class 1.
- Ridge training and test MSE versus PCA dimension $p$.

### 3.6 Results

The best models were selected using the test MSE.

| **Model** | **Best Test MSE** | **Test Accuracy** |
|---|---|---|
| Ridge | 0.051157 | 94.76% |
| Lasso | 0.051161 | 94.76% |

Table 1: Question 1 final results.

### 3.7 Analysis

1. **Ridge and lasso MSE versus $\lambda$:** For small values of $\lambda$, both ridge and lasso achieved their lowest MSE values. As $\lambda$ increased, the models became more heavily regularized and the MSE rose, showing the expected underfitting trend. The training and test curves stayed close to each other, which indicates that the PCA-reduced problem is well behaved and the models generalize well in the low-regularization regime.

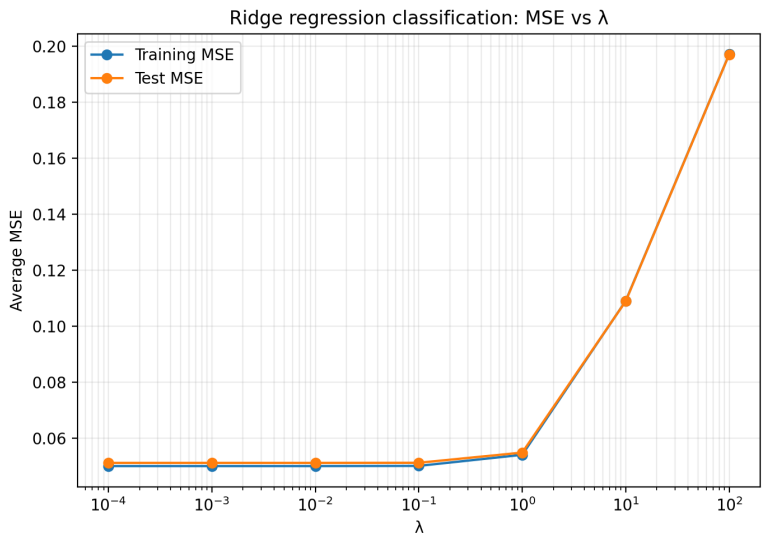


Figure 1: Ridge regression classification: MSE vs $\lambda$

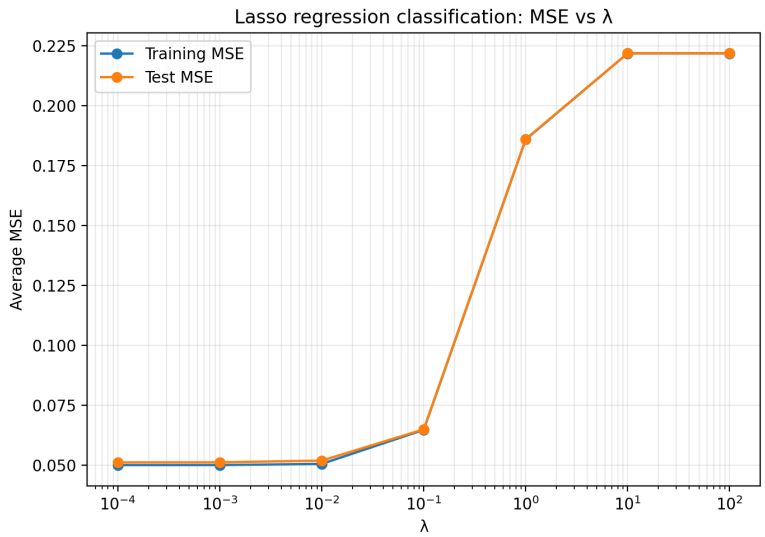


Figure 2: Lasso regression classification: MSE vs $\lambda$

2. **Lasso sparsity pattern:** As $\lambda$ increased, the number of non-zero lasso coefficients dropped sharply. This is the standard sparsity effect of the $L_1$ penalty. At large regularization, lasso shrinks most coefficients to zero.

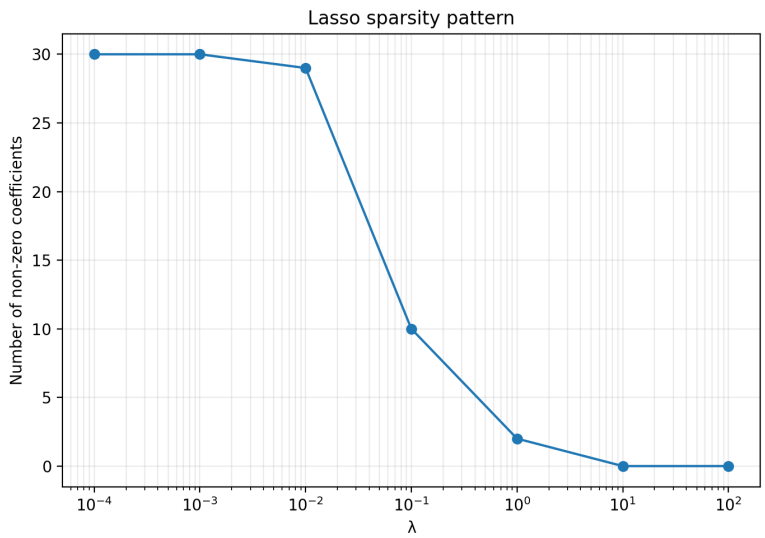


Figure 3: Lasso sparsity pattern

3. **Regularization paths:** Ridge coefficients decreased smoothly toward zero as $\lambda$ increased. Lasso coefficients were piecewise linear and many crossed exactly to zero at specific regularization values. This difference reflects the geometric difference between $L_2$ and $L_1$ regularization.

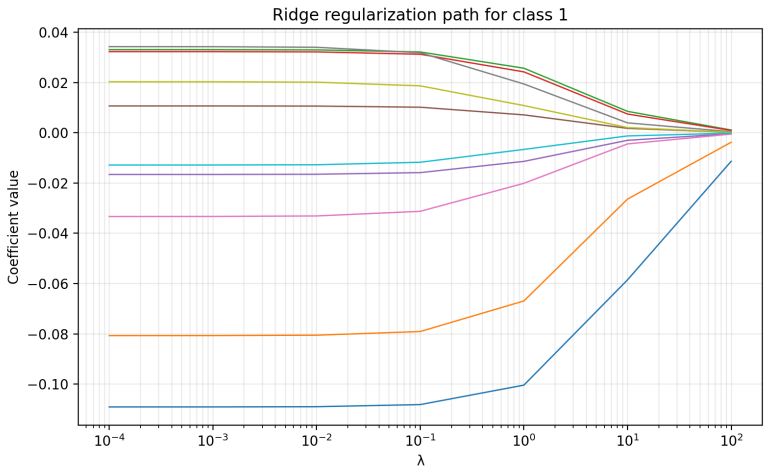


Figure 4: Ridge regularization path for class 1

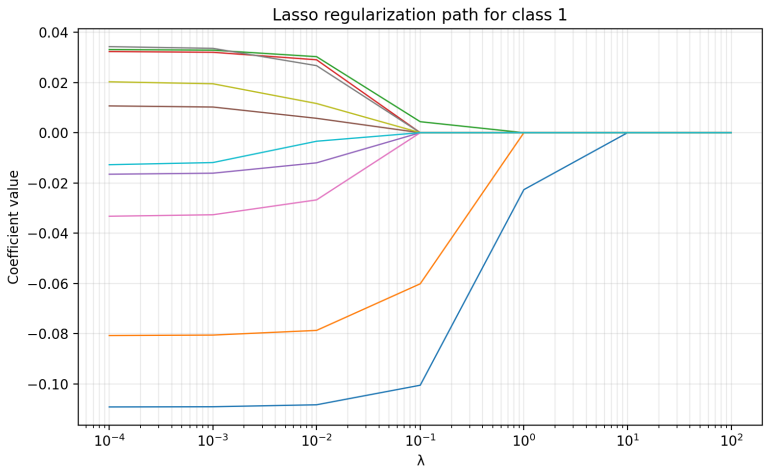


Figure 5: Lasso regularization path for class 1

4. **Complexity study:** Increasing the PCA dimension generally improved the ridge model because more informative variance from the original images was retained. The best trade-off was obtained at the selected ridge regularization value from the main experiment.

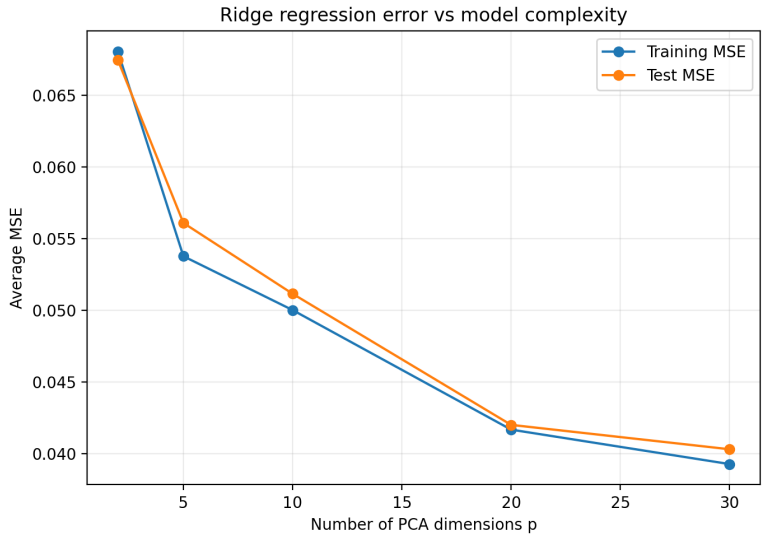


Figure 6: Ridge regression error vs model complexity

# 4 Question 2: Decision Trees, Bagging, and Random Forest

## 4.1 PCA

The MNIST data were first reduced to $p = 10$ PCA dimensions using the training subset of classes $\{0, 1, 2\}$.

## 4.2 Decision Tree Construction

A binary decision tree with exactly three terminal nodes was learned. The splitting criterion was the weighted Gini index:

$$Gini(S) = 1 - \sum_{k=1}^{K} p_k^2,$$

$$Gini_{weighted} = \sum_i \frac{|S_i|}{|S|} Gini(S_i).$$

For each split, the median of each feature was used as the candidate threshold. The feature and threshold that minimized the weighted Gini index were selected. Using the median makes the split rule deterministic and easy to justify for this shallow tree.

### 4.3 Bagging

Five bootstrap datasets were created by sampling with replacement from the training data. A three-node tree was trained on each bootstrap sample. Out-of-bag (OOB) error was computed using the training samples that were not selected in each bootstrap sample. Test prediction was done using majority voting over the five trees.

### 4.4 Random Forest

The same five bootstrap samples were reused. At each split, only $k$ randomly selected features were considered. The best split was chosen only among those features. The value of $k$ was selected by minimizing the OOB error.

### 4.5 Results

| **Method** | **Test Accuracy** | **Average OOB Error** |
|---|---|---|
| Single tree | 78.17% | – |
| Bagging | 77.53% | 0.218695 |
| Random forest | 77.53% | 0.218695 |

Table 2: Question 2 final results.

Class-wise accuracies were:

$$\text{Single tree: } \{0 : 0.7755,\ 1 : 0.9974,\ 2 : 0.5504\}$$

$$\text{Bagging: } \{0 : 0.7735,\ 1 : 0.9974,\ 2 : 0.5329\}$$

$$\text{Random forest: } \{0 : 0.7735,\ 1 : 0.9974,\ 2 : 0.5329\}$$

### 4.6 Analysis

1. **Single tree:** The single tree is simple and interpretable, but it is constrained by the requirement of only three terminal nodes. This makes it a high-bias classifier.

2. **Bagging:** Bagging reduced variance by averaging over bootstrap trees, but because the trees are extremely shallow, the gain was limited. The class-wise accuracy shows that class 1 is very easy to detect, while class 2 is the hardest.

3. **Random forest:** The chosen value of $k$ was 9, which is close to the full feature dimension $p = 10$. In this shallow-tree setting, allowing nearly all features at each split made random forest behave very similarly to bagging. This explains why the bagging and random forest results are nearly identical.

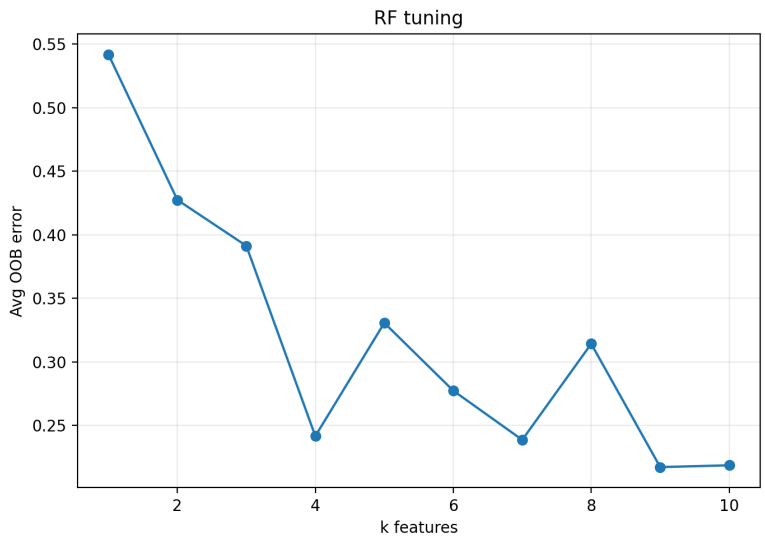


Figure 7: Random forest tuning via OOB error

# 5 Question 3: Regression Stump and Bagging on Fashion-MNIST

## 5.1 Data Preparation

Fashion-MNIST classes 0, 1, and 2 were selected. The images were flattened, normalized, and then projected onto $p = 10$ PCA dimensions using the training subset.

## 5.2 Regression Stump

The class labels were treated as numerical responses $\{0, 1, 2\}$. A decision stump was learned by searching all features and all candidate split thresholds. For a feature, the candidate thresholds were the midpoints between consecutive sorted values.

For a split into left and right regions, the sum of squared residuals (SSR) is

$$SSR = \sum_{i \in L} (y_i - \bar{y}_L)^2 + \sum_{i \in R} (y_i - \bar{y}_R)^2.$$

The split minimizing SSR was selected.

## 5.3 Bagging

Five bootstrap samples were created. A regression stump was trained on each bootstrap dataset. The bagged prediction was the average of the five stump predictions. The OOB error was computed using the samples not selected in each bootstrap dataset.

## 5.4 Results

| Method | Train MSE | Test MSE |
|---|---|---|
| Single stump | 0.345563 | 0.362944 |
| Bagged stump | 0.337698 | 0.356993 |

Table 3: Question 3 final results.

The average OOB MSE across the five bootstrapped stumps was 0.351174. The selected stump used feature 2 with threshold $-1.450797$.

### 5.5 Analysis

1. **Single stump:** A stump is a very weak model because it can only split the feature space once. This leads to a simple, step-like prediction surface and relatively high bias.

2. **Bagging:** Bagging slightly improved the performance by averaging multiple stumps trained on different bootstrap samples. This reduces variance and smooths the prediction curve.

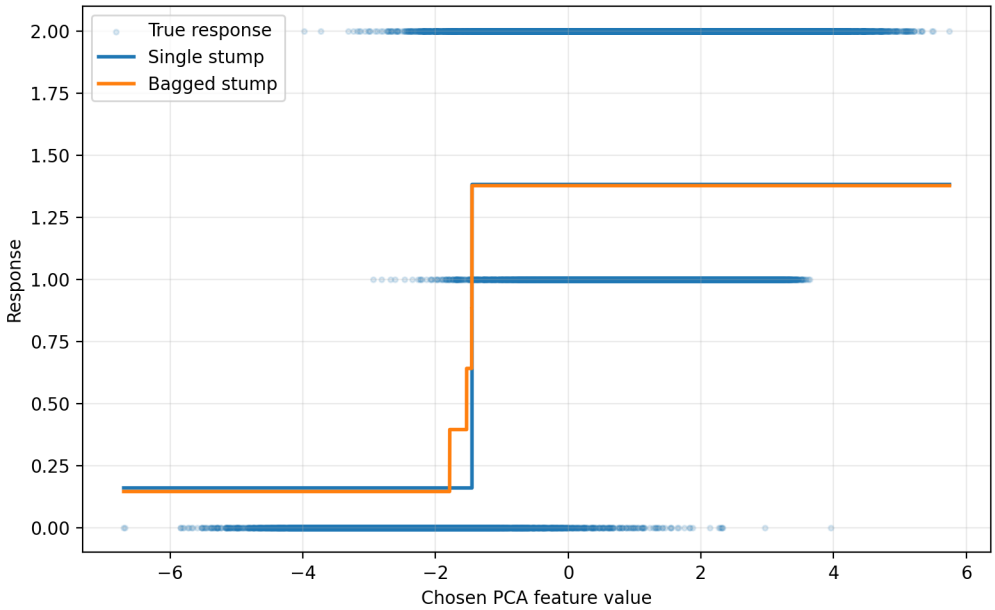


Figure 8: Decision stump vs bagging on Fashion-MNIST

3. **OOB error:** The OOB MSE is close to the test MSE, which is a good sign that the ensemble's generalization performance is being estimated reasonably well.

# 6 Conclusion

Ridge and lasso regression performed strongly on the PCA-reduced MNIST subset, with both methods producing nearly the same test accuracy. The shallow decision tree in Question 2 was interpretable but limited, and bagging/random forest gave only small changes because the tree depth was extremely restricted. In Question 3, bagging improved the regression stump slightly by reducing variance. Overall, the experiments match the expected behavior of linear models, trees, bagging, and random forests discussed in class.

# 7 How to Run

```
python 2023090_A3_Q1.py --output-dir outputs_q1
python 2023090_A3_Q2.py --output-dir outputs_q2
python 2023090_A3_Q3.py --data-dir data/fashion_mnist --output-dir outputs_q3
```